

MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 2

SERIAL COMMUNICATION

SECTION 1

GROUP 5

SUBMISSION DATE: 19 OCTOBER 2026

NO.	MATRIC NO.	NAME
1	2417497	MUHAMMAD AIDIL HAZIM BIN SHAHARUL AMIR
2	2413393	MUHAMMAD DANISH BIN ZAINUDDIN
3	2415821	MUHAMMAD DANISH HAIQAL BIN MOHD EFENDY
4	2417169	MUHAMMAD DANISH NAJWAN BIN MOHD ALI

ABSTRACT

This project investigated the implementation of a serial-based data acquisition and control system utilizing communication between an Arduino Uno microcontroller and a Python environment. The primary goal was to establish hardware-software synchronization across two core tasks: acquiring analog sensor data and driving system feedback via serial data streams. Task 1 interfaced a potentiometer and an LED with 220 Ω current-limiting resistors to stream analog voltage readings via pyserial to a Python terminal, triggering conditional LED actuation based on threshold values. Task 2 expanded this task by integrating real-time data visualization using matplotlib to plot sensor readings, along with hardware control. Embedded Arduino sketches processed analog and serial inputs, while Python scripts managed data, conditional logic, and graphical rendering. Experimental validation showed somewhat real-time data transmission, hardware actuation, and graphical tracking without serious communication bottlenecks. Overall, this integrated serial communication framework demonstrated an effective methodology for bridging physical components with software environments for monitoring and control applications.

Table of Contents

TASK 1....4
1.0 Introduction & Theory....4
1.1 Materials & Equipment....4
1.2 Experimental Setup....4
1.3 Methodology & Programming Logic....5
1.4 Data collection & analysis....6
1.5 Results & Discussion....7
1.6 Conclusion....7
TASK 2....8
2.0 Introduction & Theory....8
2.1 Materials and Equipment....8
2.2 Experimental Setup....8
2.3 Methodology & Programming Logic....9
2.4 Data collection & analysis....12
2.5 Results & Discussion....12
2.6 Conclusion....13
RECOMMENDATIONS....13
ACKNOWLEDGEMENT....14
STUDENT DECLARATION....15

TASK 1

1.0 Introduction & Theory

This experiment explores the communication between hardware and software, the use of a potentiometer, and Python serial communication. The core focus of this session is Python-based serial communication, specifically displaying the value of the potentiometer through Python, along with an additional LED turning on when the potentiometer reaches a maximum assigned value. The expected outcome is a fully functional circuit where a user can monitor the value of the potentiometer through Python while displaying a turned-on LED as it reaches the maximum value.

1.1 Materials & Equipment

- Arduino Uno
- Potentiometer
- 220 Ω resistor
- Breadboard & Jumper Wires
- Laptop with Python Environment

1.2 Experimental Setup

1. Pin connection details:
   - LED (Pin 7)
   - Potentiometer (Pin A0)

2. Hardware configuration.

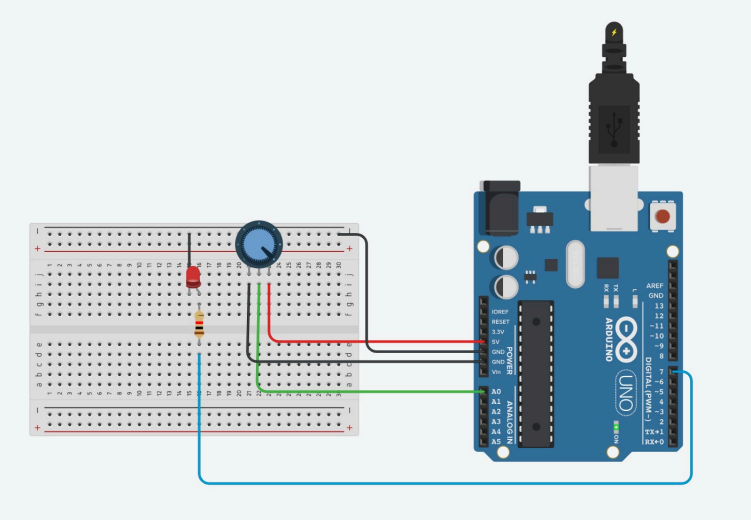


Figure 1.1: Circuit Assembly Diagram

1.3 Methodology & Programming Logic

1. Add an LED in series with a 220 Ω current-limiting resistor and connect it to pin 7. Also, connect the potentiometer to analog pin A0.
2. Connect the Arduino Uno to a laptop via a USB cable.
3. Launch the Arduino IDE and upload code to the microcontroller.
4. Close Arduino IDE before going into Python streaming.
5. Run a Python script in PyCharm to begin streaming and observing raw potentiometer values directly inside the terminal.

```
const int potPin = A0;  // Potentiometer connected to analog pin A0
const int ledPin = 7;
const int threshold = 500; // Change this value to adjust sensitivity

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int potValue = analogRead(potPin);

  // Check if potentiometer exceeds the threshold
  if (potValue > threshold) {
    digitalWrite(ledPin, HIGH); // Turn LED ON
  } else {
    digitalWrite(ledPin, LOW);  // Turn LED OFF
  }

  Serial.println(potValue);
  delay(100); // Small delay for stability
}
```

script.py

```python
import serial

# Replace 'COM7' with your actual port if it changes
ser = serial.Serial( port: 'COM7',  baudrate: 9600)

try:
    print("Reading potentiometer... Press Ctrl+C to stop.")
    while True:
        pot_value = ser.readline().decode().strip()
        if pot_value:
            print("Potentiometer Value:", pot_value)

except KeyboardInterrupt:
    ser.close()
    print("Serial connection closed.")
```

1.4 Data collection & analysis

Observation	Potentiometer Reading (0-1023)	Python Logic Evaluation	Hardware Status (LED)
1	0 (Minimum)	Value <= 500	OFF
2	367	Value <= 500	OFF
3	500 (Threshold)	Value <= 500	OFF
4	501 (Threshold Crossed)	Value > 500	ON
5	1023 (Maximum)	Value > 500	ON

The data confirmed that:

- **Successful Bidirectional Communication:** The system reliably transmitted real-time potentiometer readings (ranging from 0 to 1023) from the Arduino to the Python terminal as the knob was manually rotated.
- **Accurate Threshold Actuation:** When the potentiometer value exceeded the physical midpoint (500 and above), the Python script successfully triggered the breadboard LED to illuminate. Values below this threshold accurately extinguished the LED.

1.5 Results & Discussion

The result of the experiment successfully shows:

- The Python terminal successfully displayed real-time potentiometer values (0 to 1023) as the knob was adjusted.
- The LED illuminated instantly when the reading surpassed 500 and turned off when the value dropped below this threshold.

The discussion :

1. **Interpretation of Results:** The experiment demonstrated successful closed-loop control via PC. Offloading the decision threshold logic to Python proves that high-level software can efficiently direct microcontroller hardware, enabling more scalable and complex mechatronic control architectures.

2. **Discrepancies Between Expected and Observed:** A minor latency was occasionally observed between turning the potentiometer and the LED toggling. This slight lag is introduced by the sequential nature of reading the serial buffer, processing the string in Python, and sending a command back, compounded by the 100ms hardware delay.

3. **Sources of Error and Improvement:**

    - **COM Port Conflicts:** A frequent hurdle we ran into was the `PermissionError: Access is denied` warning in Python. Since Windows only allows one application to use a COM port at a time, we fixed this by making sure to close any active Arduino IDE serial monitors before running our scripts in PyCharm.

1.6 Conclusion

This project successfully demonstrated the design and implementation of bidirectional serial communication using an Arduino microcontroller and a Python script. The system was able to accurately read analog signals from a potentiometer and control an LED based on software-defined thresholds. After going through the process of putting the circuit together, coding it, and testing, the project achieved its main objective, which is to understand and implement hardware-software interfacing between an Arduino and high-level programming logic.

In terms of understanding, we now know how to establish communication protocols between different platforms, and as for implementation, we are confident in the reenactability of this project for more complex control applications. The results proved that even simple mechatronic systems can be effectively developed through systematic design and logical programming.

For an Islamic reflection, this project highlights the importance of seeking knowledge (Ilm') seen in our objective to understand its inner workings. Knowledge is a gift from Allah that enables mankind to understand and control the world around them. By exploring the principles of electronics, software integration, and control systems, we recognize the precision and order that Allah has placed in creation.

TASK 2

2.0 Introduction & Theory

This experiment explores serial communication between hardware and software, integrating a potentiometer, a servo motor, and an LED with an Arduino Uno. The core focus of this session is to control the angle of a servo motor in real time by turning the potentiometer knob, while simultaneously transmitting and plotting the sensor readings through Python. Additionally, an LED is configured to turn on automatically when the potentiometer reaches a specific threshold. The expected outcome is a fully functional system where a user can monitor potentiometer values and live graphs in Python while controlling the physical servo motor and LED.

2.1 Materials and Equipment

- Arduino Uno
- Servo Motor
- Pontetiometer
- LED
- 220 ohm Resistor
- Breadboard & Jumper Wires
- Laptop with Python Environment

2.2 Experimental Setup

1. Pin connection details.
   - LED (Pin 3)
   - Potentiometer (Pin A0)
   - Servo (Pin 9)

2. Hardware configuration

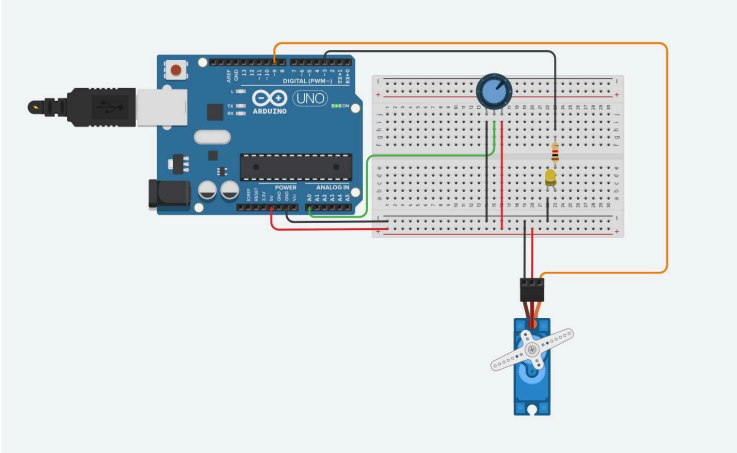


Figure 2.1: Circuit Assembly Diagram

2.3 Methodology & Programming Logic

1. Connect the potentiometer to analog pin A0, the servo motor signal to digital pin 9, and the LED in series with a 220 Ω resistor to digital pin 3.
2. Connect the Arduino Uno to a laptop via a USB cable.
3. Launch the Arduino IDE, include the Servo library, and upload the code to the microcontroller
4. Close the Arduino IDE before starting Python streaming.
5. Run the Python script in PyCharm to begin streaming potentiometer values, driving the servo motor, and observing the real-time matplotlib plot.

Arduino Idle code:

```
#include <Servo.h>

Servo myservo;
const int potPin = A0;
const int ledPin = 3;
const int threshold = 500; // Threshold where LED turns ON
bool isRunning = true;

void setup() {
  Serial.begin(9600);
  myservo.attach(9);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Check if Python sent a command to halt execution
  if (Serial.available() > 0) {
    char incoming = Serial.read();
    if (incoming == 'q') {
      isRunning = false;
```

```
    }
  }

  if (isRunning) {
    int potValue = analogRead(potPin);

    // Map potentiometer (0-1023) to servo angle (0-180)
    int angle = map(potValue, 0, 1023, 0, 180);
    myservo.write(angle);

    // Turn LED on or off based on potentiometer threshold
    if (potValue > threshold) {
      digitalWrite(ledPin, HIGH);
    } else {
      digitalWrite(ledPin, LOW);
    }

    // Send potentiometer value to Python for Matplotlib visualization
    Serial.println(potValue);
    delay(50);
  }
}
```

Pycharm code:

```
import os

# 1. Fix Tcl/Tk environment paths
os.environ['TCL_LIBRARY'] = r'C:\Program Files\Python313\tcl\tcl8.6'
os.environ['TK_LIBRARY'] = r'C:\Program Files\Python313\tcl\tk8.6'

# 2. Force Matplotlib backend BEFORE importing pyplot
import matplotlib
matplotlib.use('TkAgg')

import matplotlib.pyplot as plt
import serial

# 3. Open the serial port FIRST (Make sure Arduino IDE Serial Monitor is closed!)
ser = serial.Serial('COM5', 9600)

# 4. Set up interactive plotting
plt.ion()
fig, ax = plt.subplots()
x_vals, y_vals = [], []

(line_plot,) = ax.plot([], [], color='b', linewidth=2)

ax.set_title("Real-Time Potentiometer Readings")
ax.set_xlabel("Sample Count")
```

```
ax.set_ylabel("Potentiometer Value")

try:
    print("Plotting potentiometer values in real-time... Press Ctrl+C to stop.")
    count = 0
    MAX_POINTS = 150  # Keeps the graph fast by only showing the latest points

    while True:
        if ser.in_waiting > 0:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if line.isdigit():
                pot_value = int(line)
                print("Potentiometer Value:", pot_value)

                x_vals.append(count)
                y_vals.append(pot_value)
                count += 1

                # Trim arrays to keep only the most recent points (prevents lagging)
                if len(x_vals) > MAX_POINTS:
                    x_vals.pop(0)
                    y_vals.pop(0)

                line_plot.set_xdata(x_vals)
                line_plot.set_ydata(y_vals)

                ax.relim()
                ax.autoscale_view()
                ax.set_xlim(max(0, count - MAX_POINTS), count + 5)

                fig.canvas.draw()
                fig.canvas.flush_events()
        else:
            plt.pause(0.001)

except KeyboardInterrupt:
    ser.write(b'q')
    ser.close()
    plt.ioff()
    plt.show()
    print("Serial connection closed.")
```

2.4 Data collection & analysis

Observation	Potentiometer	Python Logic	Hardware Status	Hardware Status

	Reading (0-1023)	Evaluation	(LED)	(Servo Angle)
1	0 (Minimum)	Value <= 500	OFF	0°
2	367	Value <= 500	OFF	45°
3	500 (Threshold)	Value <= 500	OFF	90°
4	501 (Threshold Crossed)	Value > 500	ON	90°
5	1023 (Maximum)	Value > 500	ON	180° (or software limit)
6	Keyboard 'h' Pressed	Interrupt Triggered	Retain current state	Halts at current angle

The collected data confirmed the following points:

1. **Stable Bidirectional Communication:** The serial link performed reliably, successfully transmitting analog readings to the computer while sending byte commands ('1','0','h','r') back to the Arduino.
2. **Automated LED Activation:** The Python script accurately signaled the LED precisely at the midpoint of the analog scale (value 500).
3. **Mechanical Protection:** Software-enforced angle limits (5° to 170°) effectively prevented the motor from reaching its physical boundaries, avoiding power failures.

2.5 Results & Discussion

Results

The final demonstration successfully validated a complete hardware-software feedback loop:

- **Real-Time Data Visualization:** Matplotlib rendered a fluid, live-updating scrolling line graph reliably and without stability issues.
- **Immediate Feedback:** The LED reacted instantly to the Python control logic.
- **Precise Mechanical Tracking:** The servo motor faithfully mirrored the potentiometer's physical position.
- **Interactive Control:** Keyboard inputs were accurately captured to seamlessly pause and resume the motor's movement.

Discussion

1. **Interpretation of Results:** The experiment clearly showed the benefits of offloading heavy logic from the microcontroller to a computer. Doing this makes building responsive user interfaces and real-time graphical displays much easier and far more efficient.

2. **Expected vs. Observed Outcomes:** At first, the servo motor suffered from heavy vibration and jitter instead of holding a steady position. We easily fixed this by adding a software deadband to the Arduino code, ensuring the motor only responds to meaningful changes in the sensor input rather than reacting to minor noise.

3. Sources of Error & System Improvements:

- **COM Port Conflicts:** A frequent hurdle we ran into was the `PermissionError: Access is denied` warning in Python. Since Windows only allows one application to use a COM port at a time, we fixed this by making sure to close any active Arduino IDE serial monitors before running our scripts in PyCharm.
- **Preventing Power Drops:** Pulling too much current from the laptop's USB port during extreme movements caused instability in the setup. Limiting the servo's operating range to a safer window (from 5° to 170°) completely solved those power fluctuation issues.

2.6 Conclusion

This laboratory experiment successfully demonstrated reliable, two-way serial communication between an Arduino and a Python app. By building the circuit and writing custom code, we were able to plot live sensor data on a computer screen while controlling a motor in real time using keyboard inputs.

The lab offered great hands-on practice with handling data streams, managing execution delays, and working with serial ports. We also tackled real-world hardware issues like signal noise causing servo jitters by implementing software fixes like deadbands.

This exercise highlights the Islamic imperative to seek knowledge and appreciate the thoughtful design behind complex systems. Building physical circuits calls for strict attention to detail and smooth communication between components. By studying how digital logic interacts with physical actuators, we get to observe natural laws that reflect the order established by Allah in the universe. Ultimately, picking up these engineering skills empowers students to innovate and create technologies that genuinely benefit society.

RECOMMENDATIONS

Use a Separate Power Supply: Standard servos pull a lot of current during high-demand spikes. Drawing power straight from your Arduino's 5V pin can trip the safety cutoffs on your computer's USB port. Powering the motor with an independent battery pack or external module will keep the whole setup running smoothly and reliably.

Boost the Serial Baud Rate: Sticking to the default 9,600 baud rate creates a major bottleneck. Bumping up the communication speed to 115,200 will drastically speed up data transfer, shrinking the lag between turning the potentiometer and seeing the graph update live on the screen.

REFERENCES

Last Minute Engineers. (*n.d.*). *How servo motor works & interface it with arduino*. Last Minute Engineers. Retrieved October 12, 2026, from https://lastminuteengineers.com/servo-motor-arduino-tutorial/

Arduino. (*n.d.*). *Tutorials - From beginner to advanced*. Arduino Official Documentation. Retrieved October 12, 2026, from https://docs.arduino.cc/tutorials/

Instructables. (*n.d.*). *Arduino servo motors*. Instructables Circuits. Retrieved October 12, 2026, from https://www.instructables.com/Arduino-Servo-Motors/

APPENDICES

Video link to our group work :

- https://vt.tiktok.com/ZSgMPbuvM/
- https://vt.tiktok.com/ZSgMPTLob/

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the report by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the report is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: Muhammad Aidil Hazim bin Shaharul Amir Matric No: 2417497	Read [✔] Understand [✔] Agree [✔]
	Name: Muhammad Danish bin Zainuddin Matric No: 2413393	Read [✔] Understand [✔] Agree [✔]
	Name: Muhammad Danish Haiqal bin Mohd Efendy Matric No: 2415821	Read [✔] Understand [✔] Agree [✔]
	Name: Muhammad Danish Najwan bin Mohd Ali Matric No: 2417169	Read [✔] Understand [✔] Agree [✔]